InnPoint Hotel System Documentation
MAS

Shehabeldin Mohamed

Table Of Contents

Contents

User Requirements

System Overview

The **Hotel InnPoint Management System** is a desktop application designed to support the daily operations of Hotel InnPoint. The system serves five roles: customers, receptionists, housekeepers, technicians, and administrators. It manages room reservations, the check-in and check-out process, room readiness, and maintenance tracking.

User Stories

Customer

As a customer, I want to:

1. Search for available room types by selecting a start and end date range, so that I can see all options that have at least one physical room free for my chosen period.
2. View the details of each available room type, including name, price per night, maximum capacity, and description, so that I can compare options and choose what suits my stay.
3. Add one or more room types to my reservation and specify the number of guests per room type, so that I can book multiple room types in a single reservation.
4. Be prevented from booking a room type that has no available rooms for my dates, so that I am never given a false confirmation.
5. Be notified when no room types are available for my chosen dates and be prompted to select different dates, so that I can complete my booking without confusion.
6. Preview the total price and the loyalty points I would earn before finalising my reservation, so that I can make an informed decision before committing.
7. Pay for my reservation using a credit card, so that I can complete my booking immediately.
8. Pay for my reservation by redeeming my loyalty points balance, so that I can benefit from points I have earned from previous stays.
9. Have my reservation confirmed immediately upon successful payment and my loyalty points balance updated accordingly, so that I know my booking is secured.
10. View all my existing reservations alongside the room types booked in each, so that I can keep track of my upcoming and past stays.

11. Check the current status of any reservation (Pending, Confirmed, CheckedIn, Completed, or Cancelled), so that I am always informed about my booking.
12. Cancel a reservation that is in Pending or Confirmed status, so that I can free up the booking when my plans change.
13. Edit my personal information including name, email address, and password, so that my profile remains accurate and up to date.
14. View my current loyalty points balance and a history of points earned per reservation, so that I always know how many points I have available.

Receptionist

As a receptionist, I want to:

1. View all registered customers and their associated reservations in a two-panel list, so that I can quickly locate the reservation I need to act on.
2. Look up a reservation and verify its status before proceeding with check-in, so that I can confirm the guest is eligible to check in.
3. Assign a specific physical room to each booked room type during check-in, choosing from rooms currently available for that type, so that the guest is properly accommodated.
4. Be prevented from assigning a room that is not clean or has an unresolved maintenance issue, so that guests are never placed in an unprepared room.
5. Have the reservation status automatically updated to CheckedIn and the assigned rooms marked as occupied upon successful check-in, so that the system reflects the current state accurately.
6. Complete a reservation during check-out so that the guest's stay is finalised in the system.
7. Have the room status automatically updated to dirty after check-out, so that housekeeping is immediately aware the room requires attention.
8. Cancel a reservation on behalf of a customer if needed, so that I can handle booking changes at the front desk.

Housekeeper

As a housekeeper, I want to:

1. View the list of rooms assigned to me for cleaning, so that I know which rooms are my responsibility.
2. Update the cleaning status of a room as I work on it (Dirty, BeingCleaned, Clean), so that the front desk is informed of room readiness in real time.
3. Mark a room as fully cleaned once I am done, so that it can be cleared for guest assignment.
4. Have the system block room assignment for any room that is not marked as Clean, so that no guest is ever placed in an unprepared room.

Technician

As a technician, I want to:

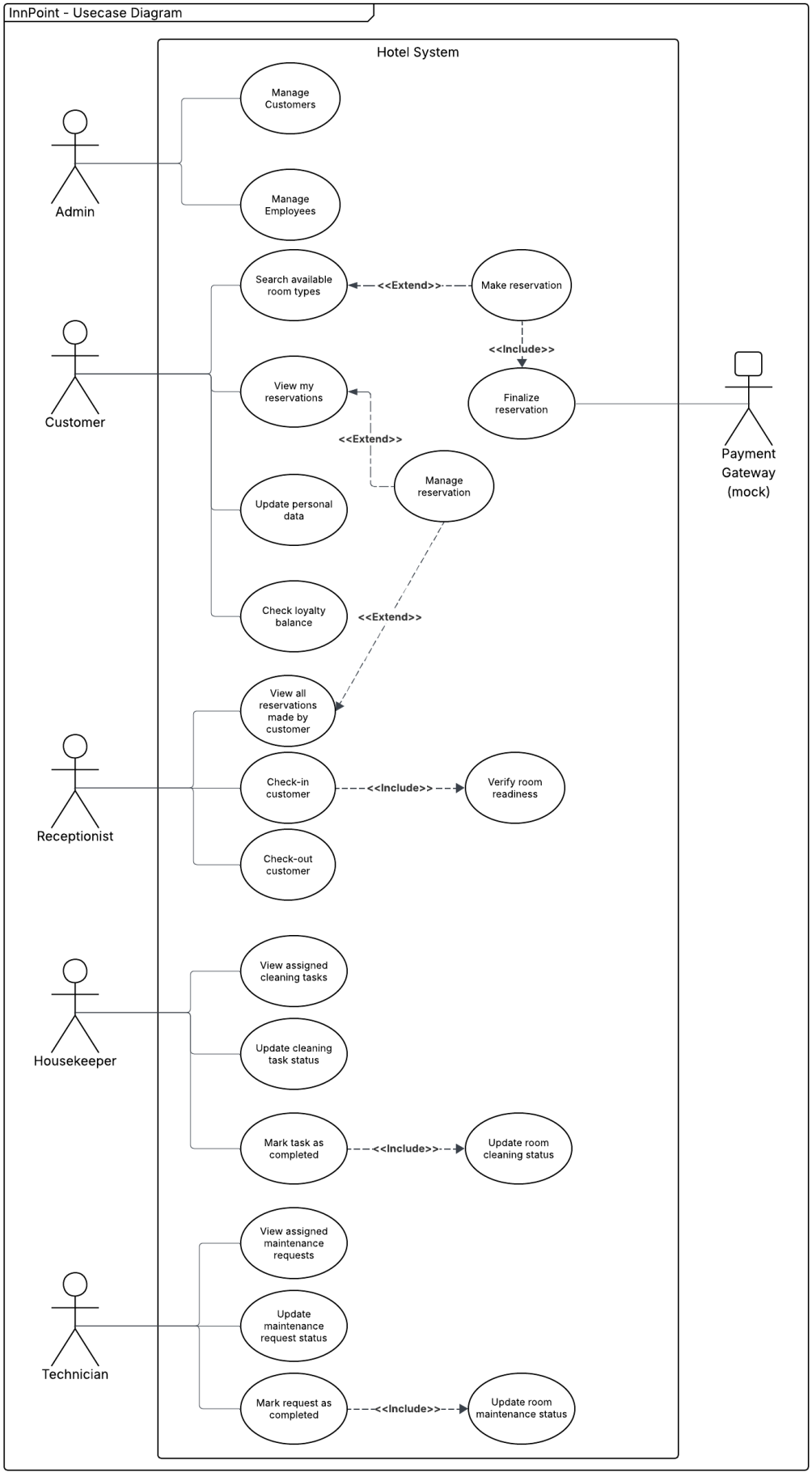
1. View all maintenance requests assigned to me, so that I know which issues require my attention.
2. Update the status of a maintenance request as I work on it, so that progress is tracked in the system.
3. Mark a maintenance request as completed once the issue is resolved, so that the room can be cleared for guest use again.
4. Have the system block room assignment for any room with an unresolved maintenance issue, so that guests are never placed in a room with a known problem.

Admin

As a admin, I want to:

1. Create new customer accounts by entering their personal details and credentials, so that customers can be registered in the system.
2. Delete a customer account from the system, so that inactive or incorrect records can be removed.
3. Create new employee accounts of type Receptionist, Housekeeper, or Technician — including their employment type, contract details, and role-specific attributes — so that staff can access the system under the correct role.
4. Delete an employee account from the system, so that former staff no longer have access.

Use case diagram



Non-Functional Requirements

Usability

The application interface must be intuitive and consistent across all user roles, following basic usability principles — clear navigation, feedback on actions, and prevention of invalid inputs.

Data Persistence

All data must be persisted between sessions using Hibernate ORM backed by an H2 relational database.

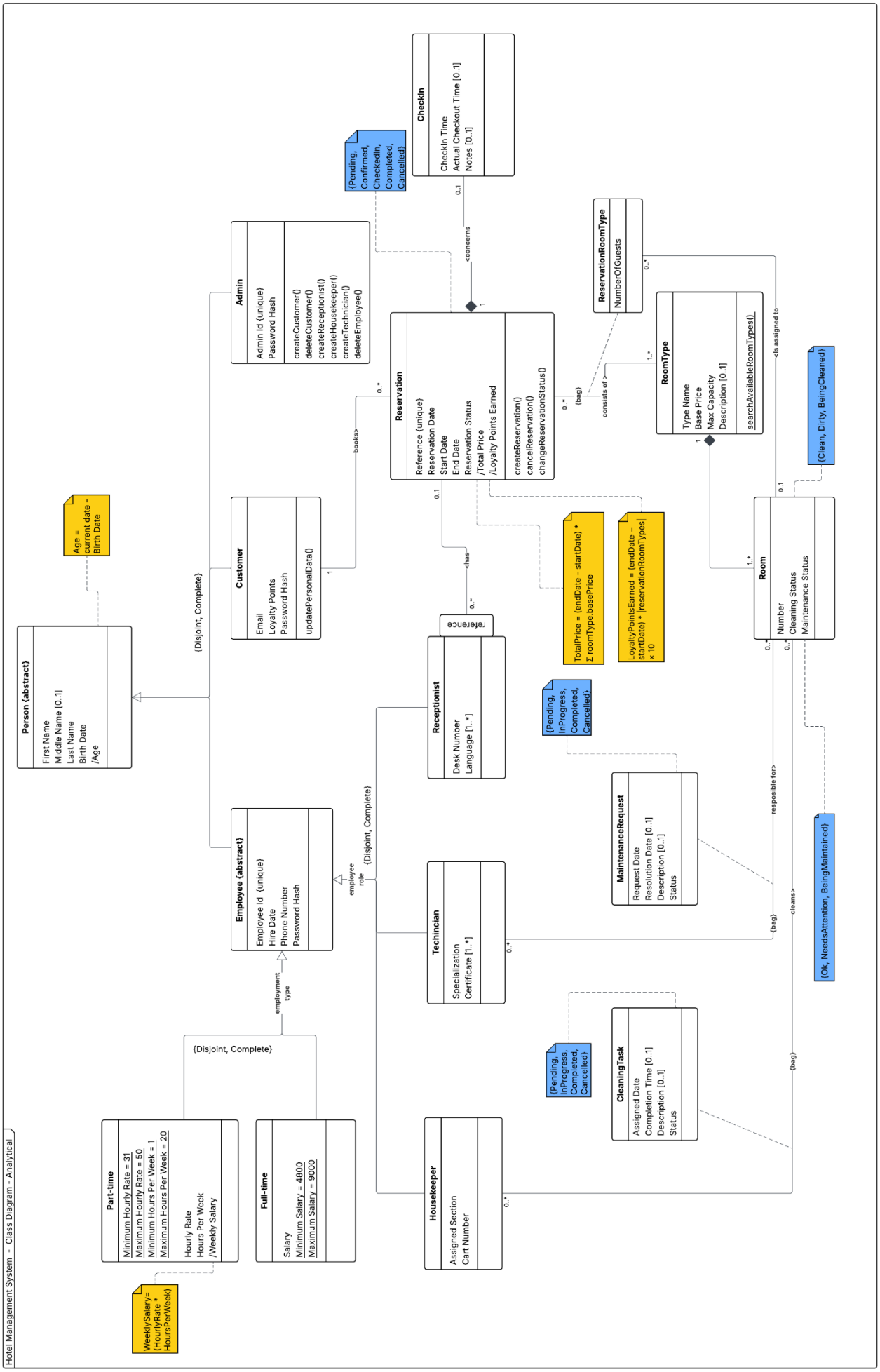
Data Integrity

The system must enforce all business rules at the domain level. Invalid states must be prevented.

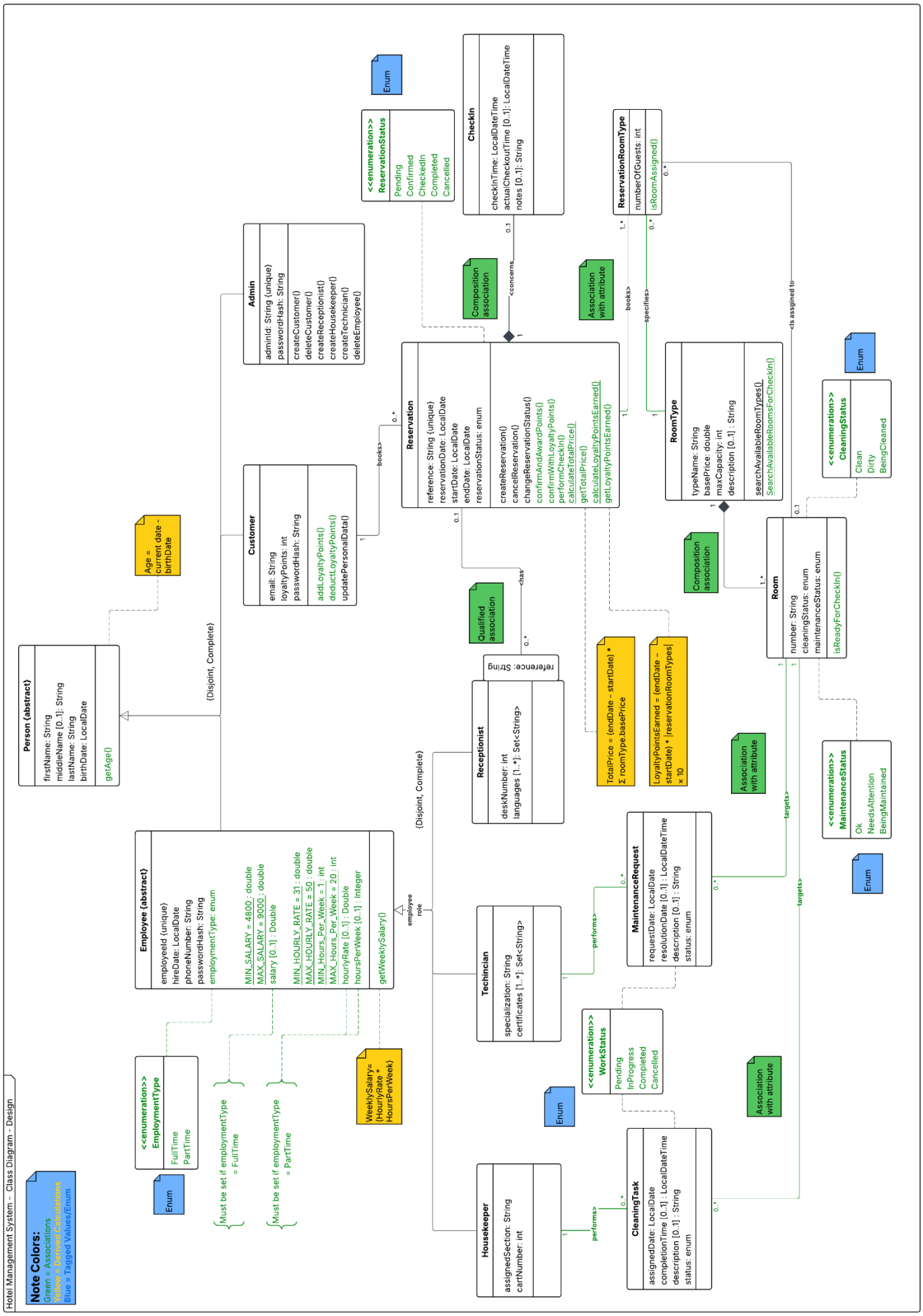
Role Separation

Each user role (customer, receptionist, housekeeper, technician, admin) must only have access to the functionality relevant to their role.

Analytical class diagram



Design class diagram



Use case scenario

Make reservation - Use case scenario

Actor: Customer
General Purpose: Book one or more room types for a specific date range.
Assumptions:
1-The customer is authenticated.
2-The system uses a mocked Payment Gateway.
Preconditions: The customer is viewing search results with at least one available room type.
Trigger: The Customer selects a room type from the search results.
Basic Flow of Events
1. System prompts for the number of guests for the selected room.
2. Customer enters the guest count and submits.
3. System displays the reservation summary (pre-filled account info, selected rooms, totals) and action options.
4. Customer clicks "Finalize Reservation".
5. System creates a "Pending" Reservation, temporarily locking the selected room inventory.
6. System displays payment form.
7. Customer selects a payment method, enters their details, and submits.
8. System processes the mock payment successfully and updates the Reservation to "Confirmed".
9. System calculates and awards loyalty points based on the booking.
10. System displays a confirmation screen (Reservation ID, summary).

Alternative Flow of Events
3a. [Customer chooses to add another room type]
- The Customer clicks the "Add Another Room Type" button.
- The system returns the Customer to the search results.
- The flow resumes from Step 1 of the Basic Flow.

3b. [Customer removes a room type]
- The Customer clicks "Remove" next to a room in the reservation summary.
- The system removes the entry and recalculates the total price and loyalty points.
- The flow resumes at Step 3.

3c. [Customer returns to search]
- The Customer clicks the "Back to Search" button.
- The system returns the Customer to the available room types list without saving.
- End of flow.

4a. [No room types selected]
- The Customer removes all rooms and attempts to click "Finalize Reservation".
- The system displays: "Please select at least one room type before finalizing."
- The flow resumes at Step 3.

7a. [Customer selects Credit Card]
- Customer fills in Cardholder Name, Card Number, and CVV.
- Flow continues.

7b. [Customer selects Loyalty Points]
- Customer chooses to pay using their accumulated loyalty point balance.
- System verifies the balance is sufficient.
- Flow continues (skipping external gateway processing in Step 8).

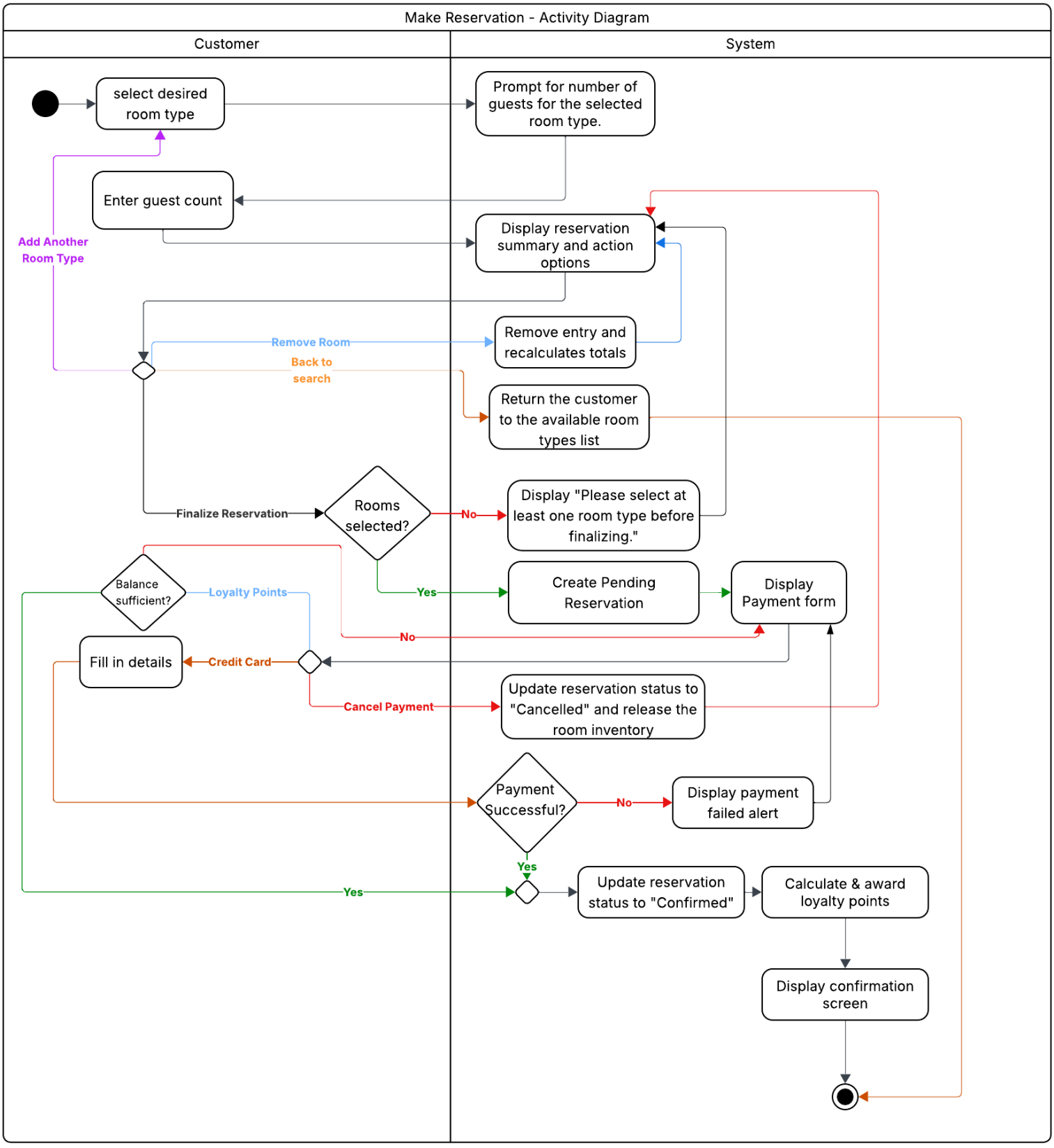
7bb. [Insufficient Loyalty Points]
- The system determines the customer’s loyalty points is insufficient to book.
- The system displays an “Insufficient loyalty points” message.
- The flow returns to step 6.

7c. [Customer cancels payment]
- The Customer clicks the "Cancel Payment" button on the payment window.
- The system updates the Pending Reservation status to "Cancelled" and releases the room inventory.
- The flow returns to Step 3.

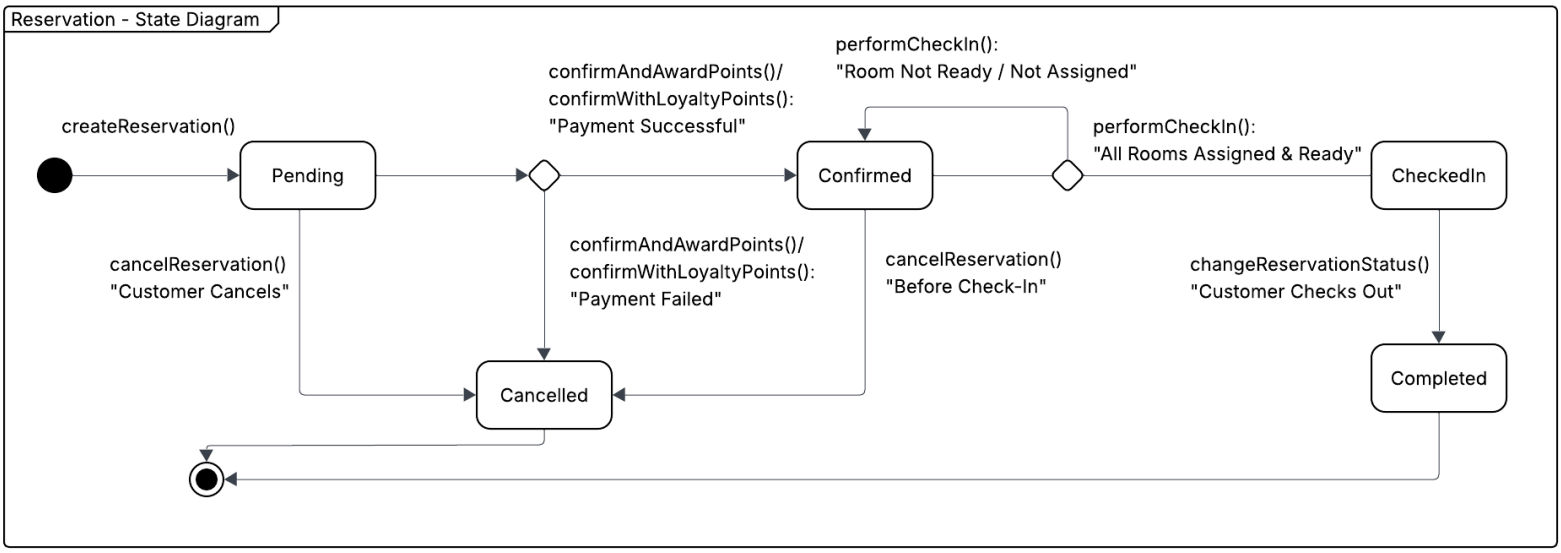
8a. [Payment simulation triggers rejection]
- The Payment Gateway mock declines the transaction.
- The system displays a "Payment failed! Please check your details and try again" alert.
- The flow returns to Step 6.

Post Conditions
- **Success:** A "Confirmed" Reservation entity is created and mapped to the Customer and Room Types. Loyalty points are awarded.
- **Cancellation :** A Reservation exists but is marked as "Cancelled", leaving the total room inventory unchanged. No points are awarded.

Activity Diagram



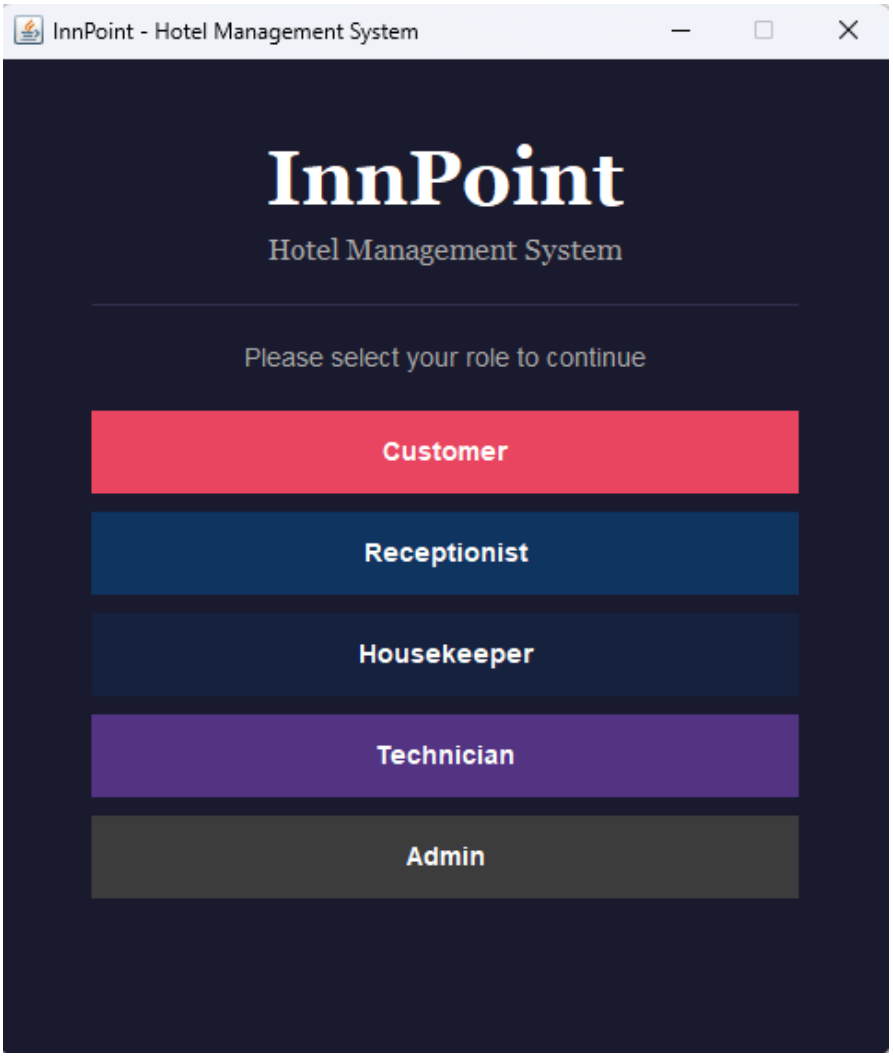
State Diagram



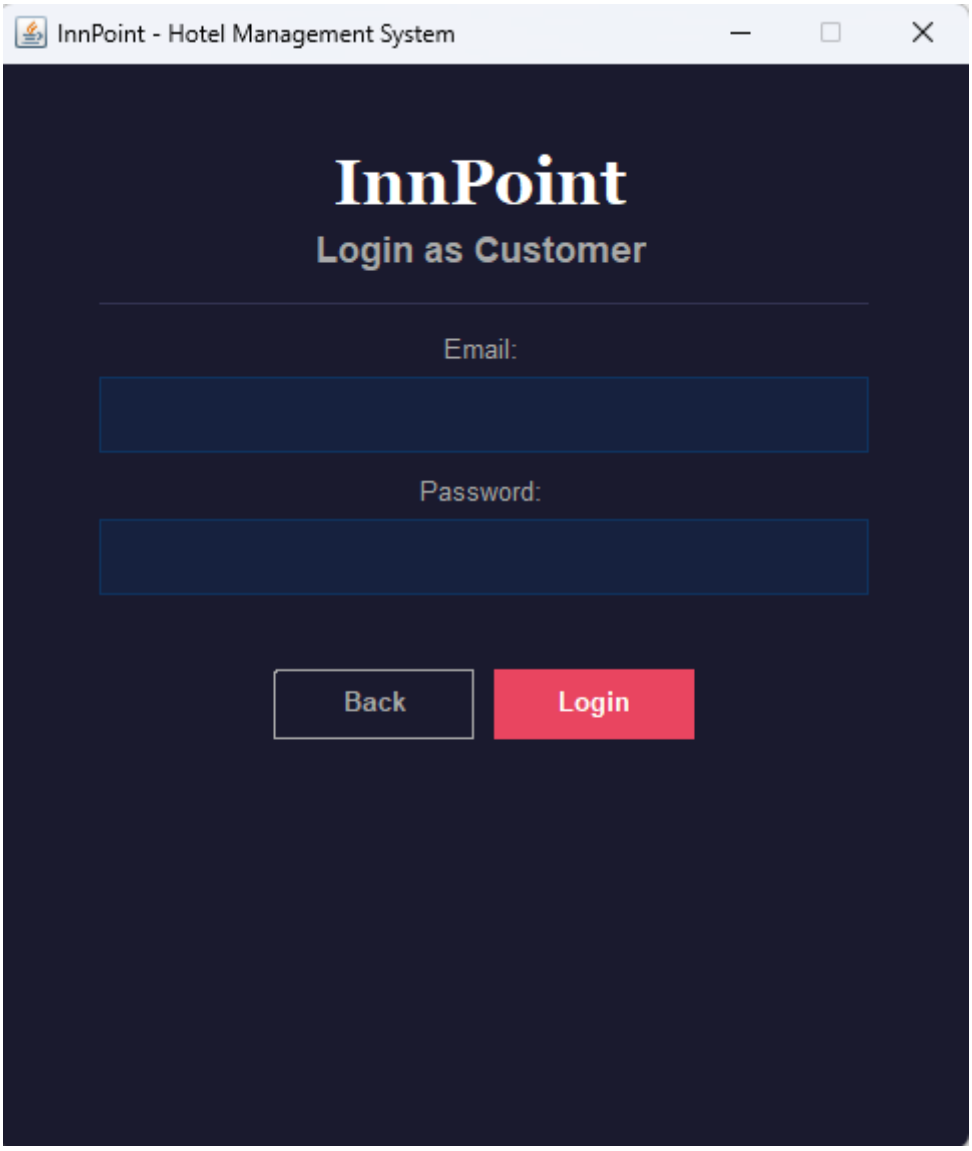
GUI design

The *InnPoint* user interface is built using Java Swing and follows a role-based navigation model. Each user role (Customer, Receptionist, and Admin) is presented with a dedicated dashboard upon login, exposing only the functionality relevant to that role.

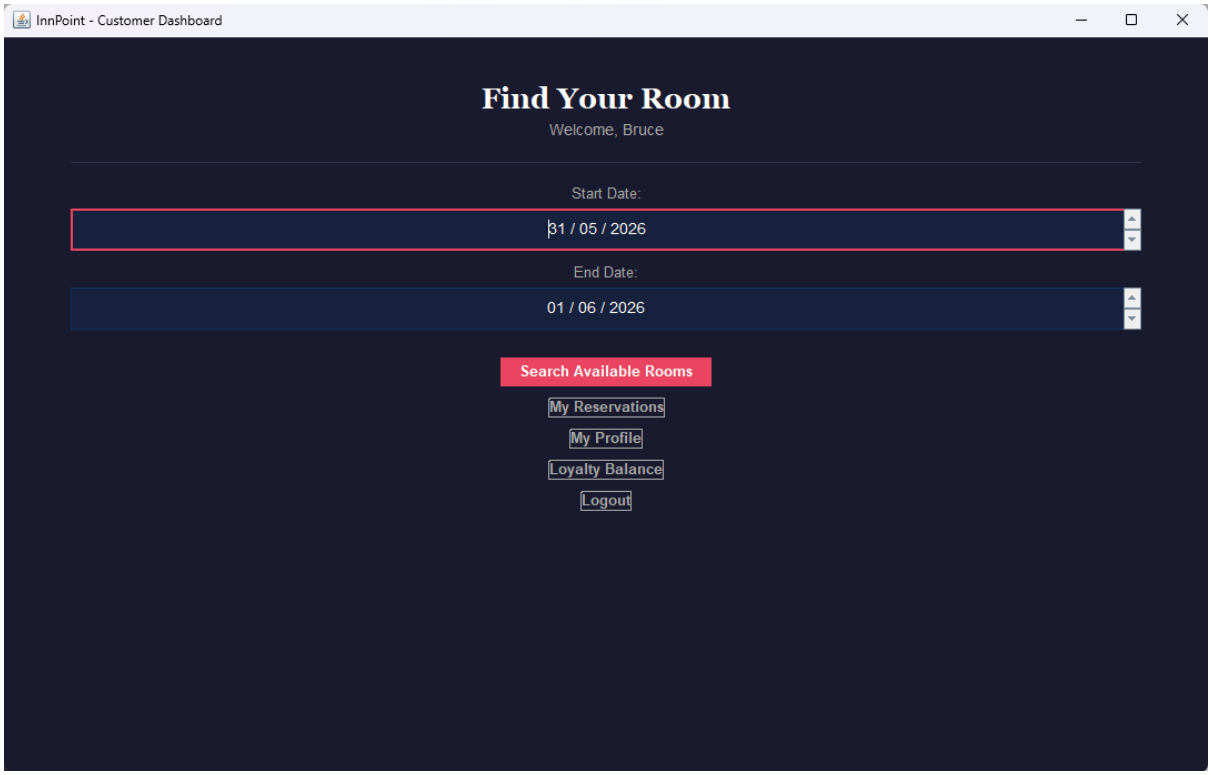
Choosing which role to login as:



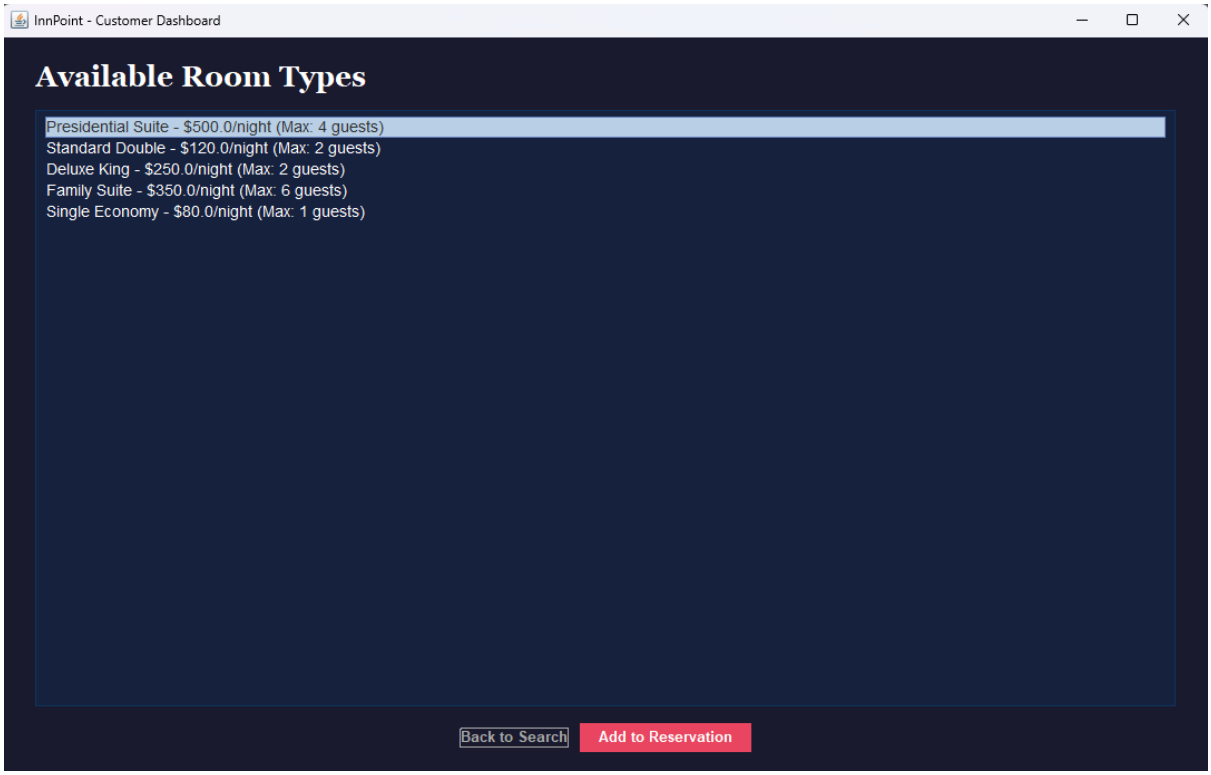
Logging in as a customer:



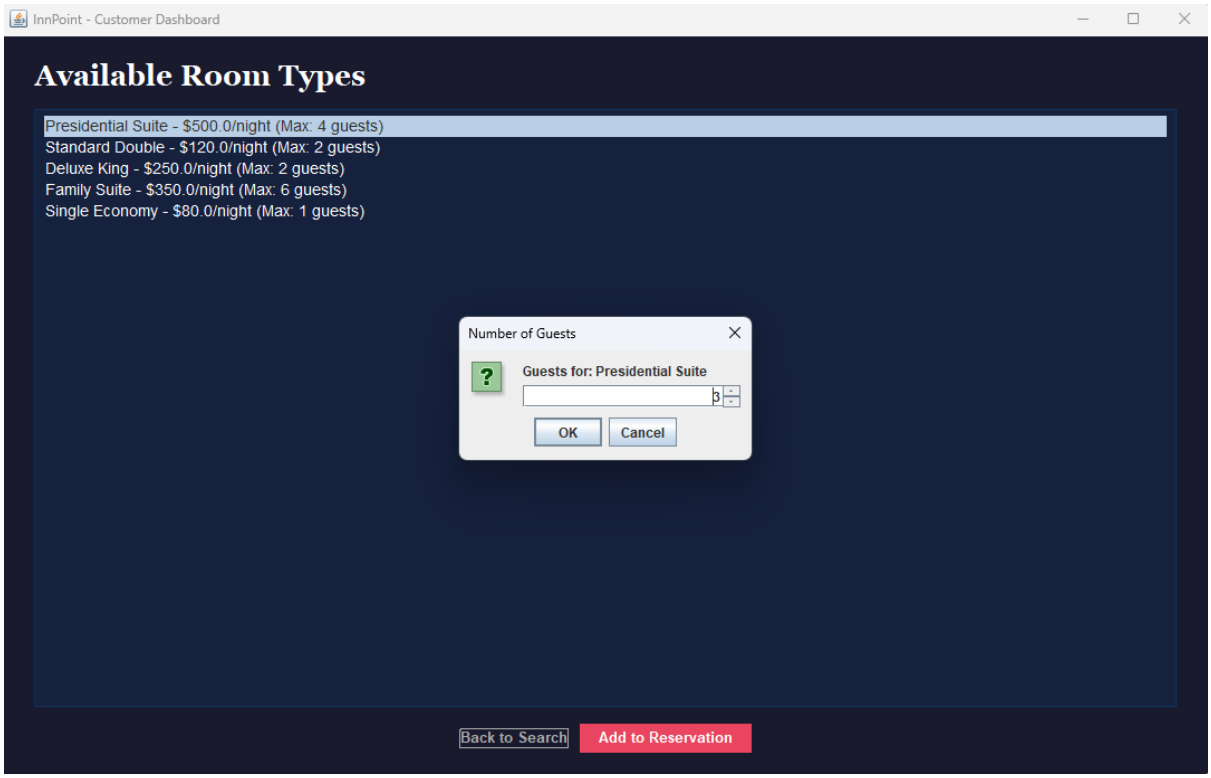
Customer Dashboard:



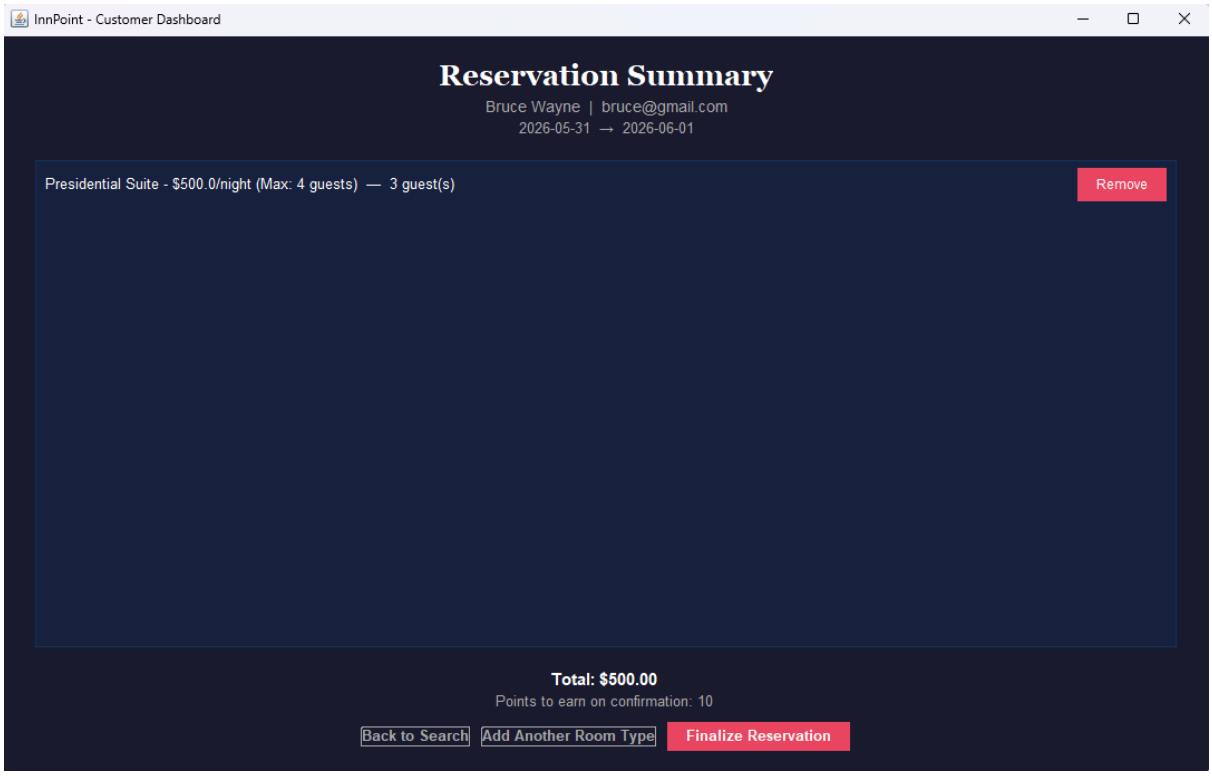
Booking a reservation as a customer (Searching for available room types):



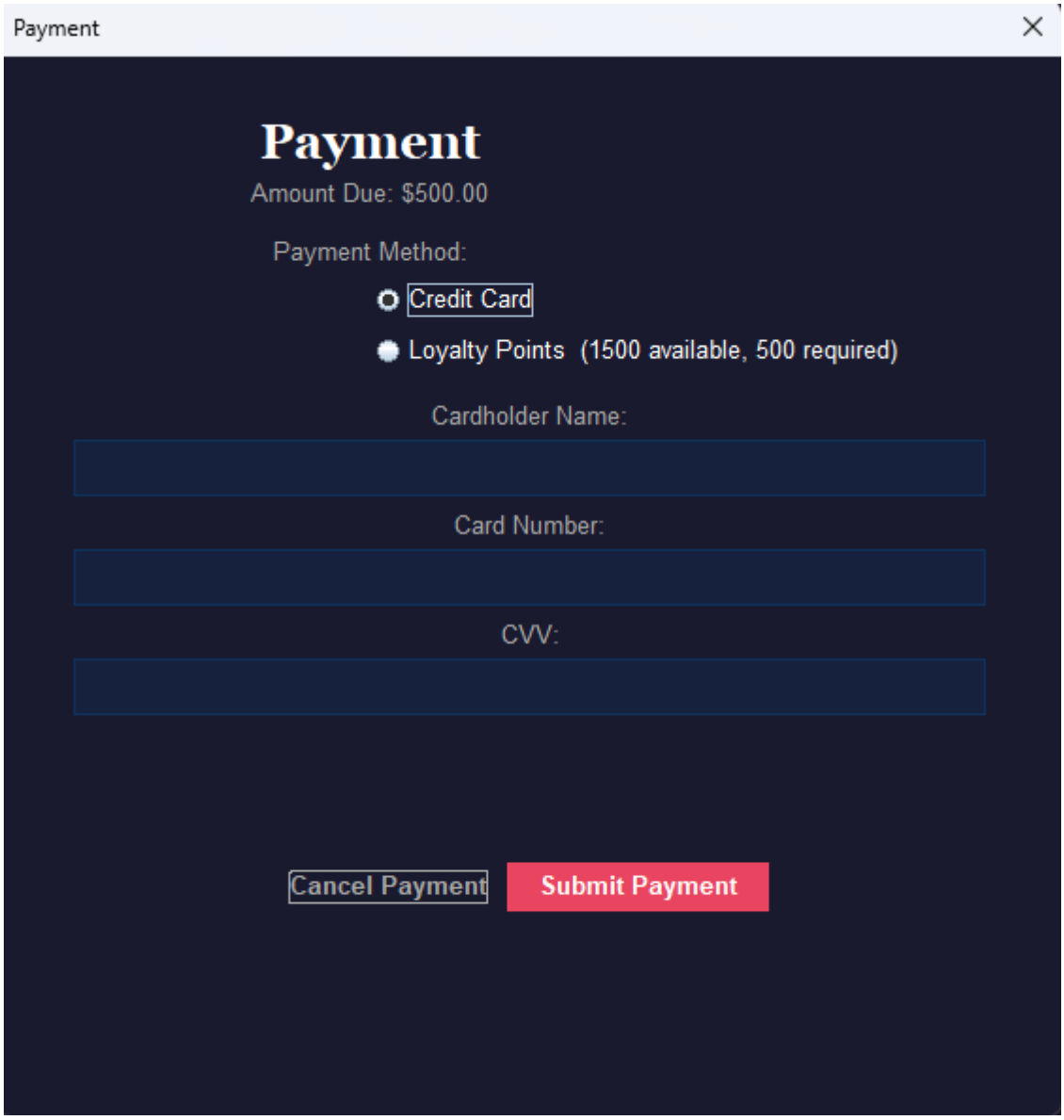
Prompting user to enter guest count:



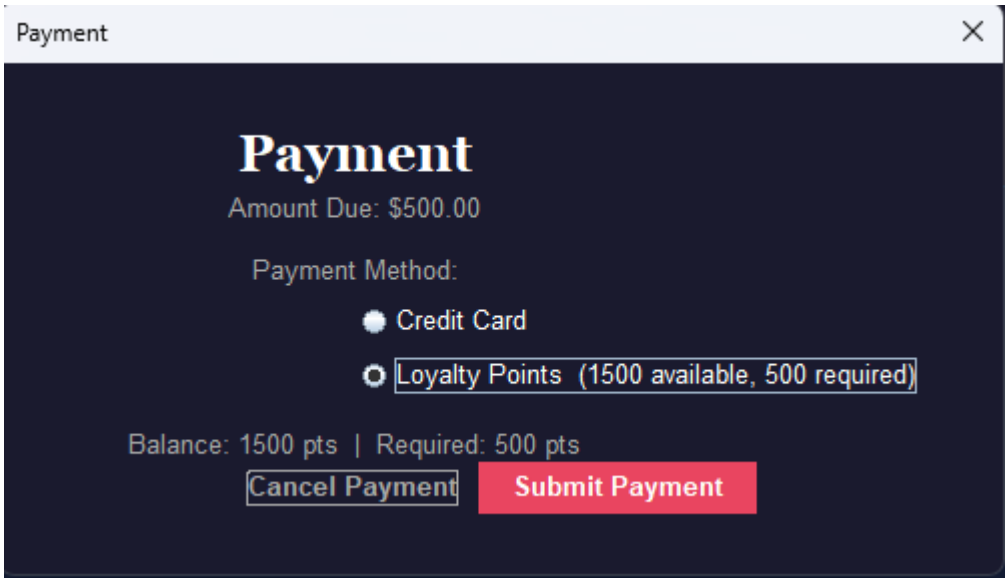
Reservation summary page:



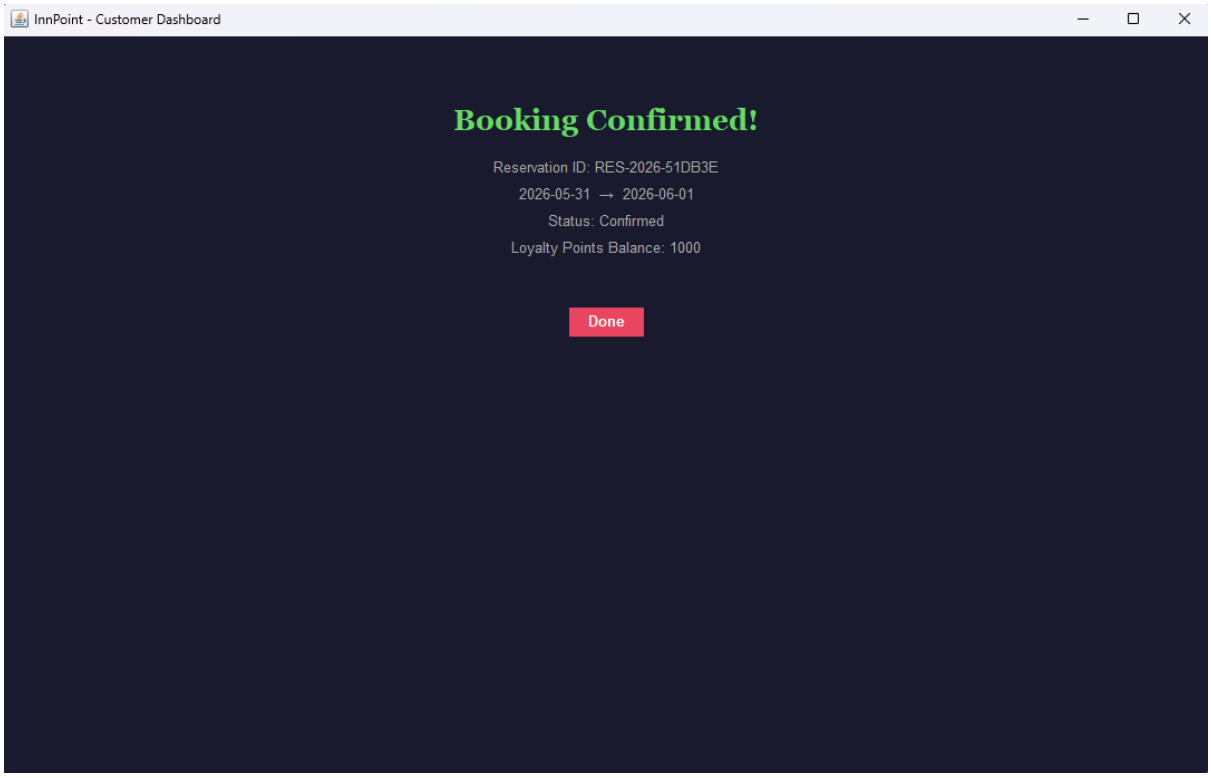
Finalizing the reservation and paying with a card:



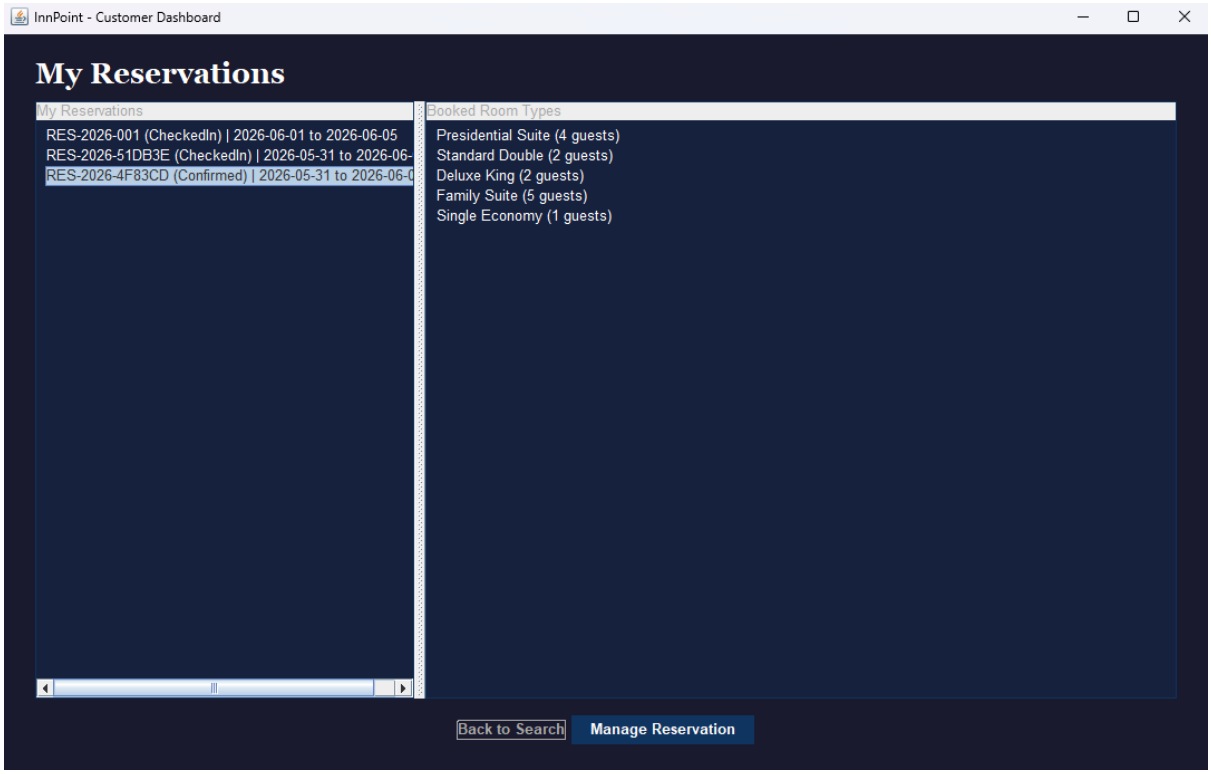
Finalizing the reservation and paying with loyalty points:



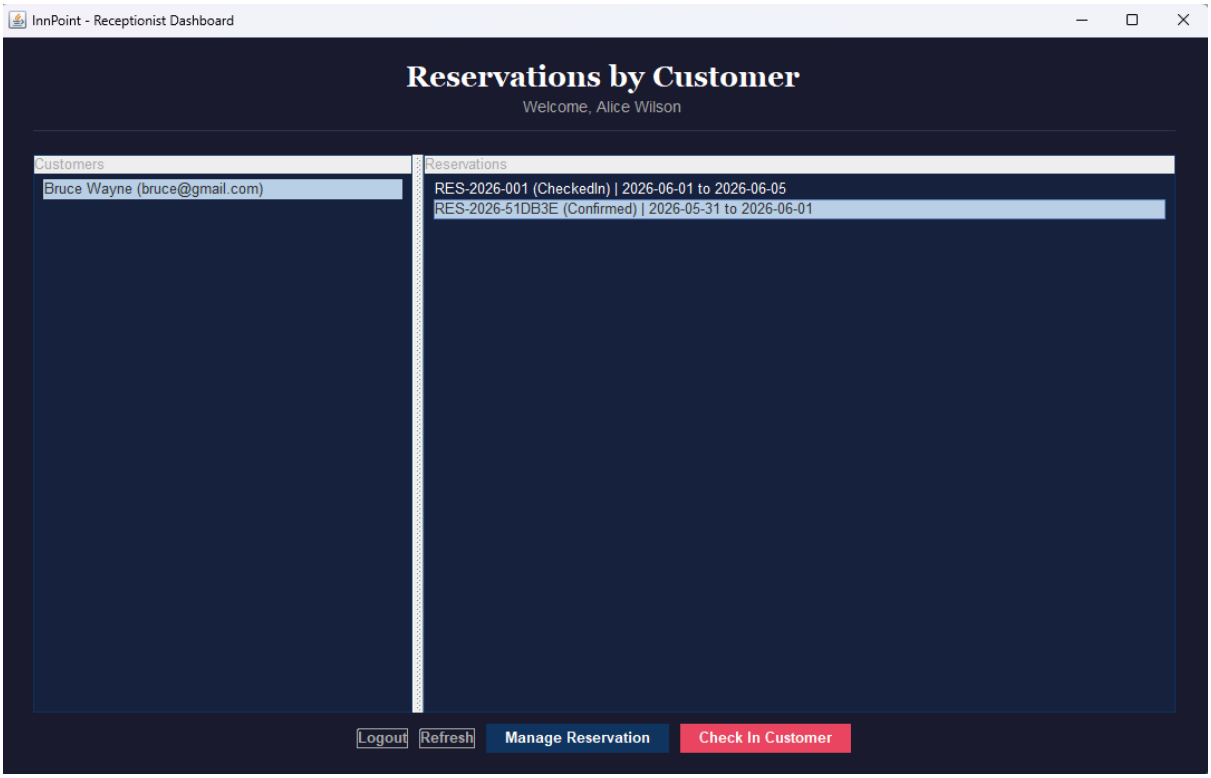
Reservation confirmation:



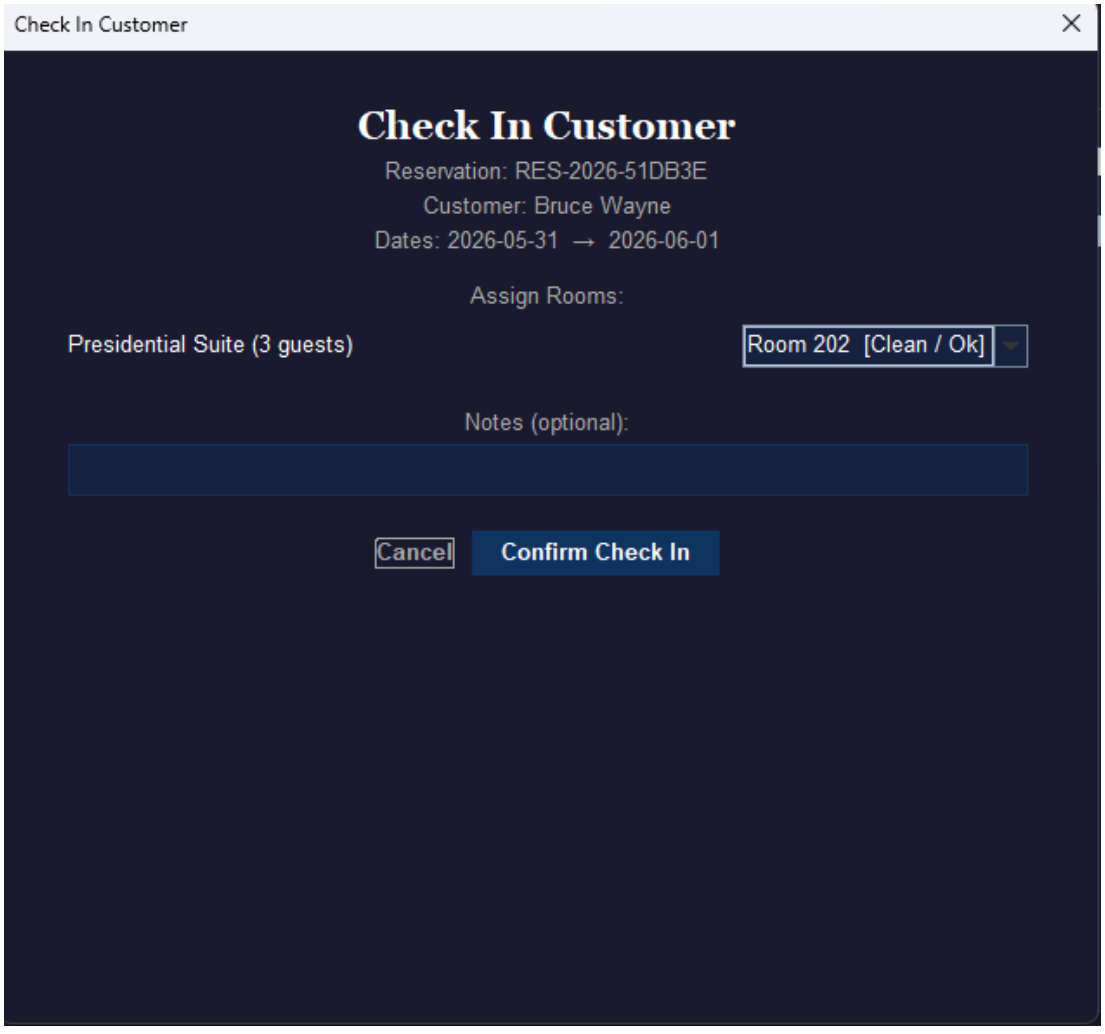
Viewing reservations and room types in that reservation as customer:



Viewing customers and their reservation as a receptionist:



Checking-In a customer as receptionist:



Design decisions and Dynamic analysis

Design Decisions

To translate the UML concepts into a concrete implementation, I made the following implementation decisions regarding attributes, inheritance, and constraints:

1. Status Attribute Mapping via Enums

The system defines multiple enumerations. All are stored with @Enumerated(EnumType.STRING), which persists the enum constant names as a readable string rather than an integer. This ensures the database values remain human-readable and are not silently broken if enum constants are reordered in the future.

2. Disjoint Inheritance for the Person Hierarchy Strategy

The **Person** class is the root of an inheritance hierarchy that includes Customer, Admin, and the abstract Employee (further extended by Receptionist, Housekeeper, and Technician). I chose JOINED inheritance strategy (@Inheritance(strategy = InheritanceType.JOINED) on Person so that each subclass stores only its own attributes in a dedicated table, linked to the Person table by a shared primary key, avoiding the wide, sparse tables produced by SINGLE_TABLE and the full duplication caused by TABLE_PER_CLASS, and keeps the schema normalised.

3. Multi-Aspect Inheritance Implementation (Flattening)

The **Employee** class carries two aspects of an employment contract: FullTime (with a fixed monthly salary) and PartTime (with and hourly rate and weekly hours). The contract fields were flattened directly into Employee with the active aspect tracked by the **EmploymentType** enum exclusion enforced in the setters. A FullTime employee must have a **salary** and cannot have **hourlyRate** or **hoursPerWeek**; a PartTime employee must have **hourlyRate** and **hoursPerWeek** and cannot have **salary**. All three fields are marked.

4. Association class and Bag Constraint Implementation

When a many-to-many relationship carries additional attributes about the relationship, an association class is needed. The system has multiple many-to-many relationships; The relationship between reservation and room type is an example, I implemented this by transforming the association into an association class, ReservationRoomType. To fulfil the bag constraint, I configured standard @OneToMany relationships from both room and roomtype down to the ReservationRoomType class.

5. Composition Constraint Via Cascades

Two composition relationships exist in the system, both implemented in Hibernate using cascade = CascadeType.ALL and orphanRemoval = true, which ensures that child objects are persisted and deleted together with their parent.

CheckIn has no existence or meaning outside its parent Reservation. It has no associations to any other entity, its constructor is private, and it can only be instantiated through Reservation.createCheckIn(). Because of this tight coupling, CheckIn is implemented as a nested static class inside Reservation, making the ownership explicit in code structure as well as in the diagram.

a Room cannot exist without belonging to a RoomType (@ManyToOne(optional = false)), and deleting a RoomType cascades to its rooms. However, unlike CheckIn, Room is a complex independent entity referenced from multiple other classes (ReservationRoomType, CleaningTask, MaintenanceRequest) with its own status attributes and business logic. For this reason it is implemented as a top-level entity class rather than a nested class, while the composition is still correctly expressed through the Hibernate cascade settings.

6. Qualified Association Implementation

The Receptionist class manages a qualified association to Reservation, where the qualifier is the reservation's unique reference number. This is implemented as a Map<String, Reservation> annotated with @OneToMany and @MapKey(name = "reference"), allowing direct lookup by reference without scanning a list.

7. Multi-Valued Attributes

Multi-valued attributes such Receptionist.languages is a Set<String> stored in a Receptionist_Languages join table, with the [1..*] multiplicity constraint enforced by @NotEmpty, and use @ElementCollection with @CollectionTable, avoiding the need to promote these simple value sets into standalone entity classes.

8. Derived Attributes

Derived attributes such as /total price is implemented as a getter method. All derived attributes are marked with @Transient to prevent Hibernate from saving them to database and attempting to map them to columns.

Dynamic Analysis

After finishing the Use Case Diagram, I performed Dynamic Analysis of the resulting design and decided to make a few additions to the functionality of the system.

1. Dual-Path Confirmation with Loyalty Points Integration

Methods: Tracing the Make a Reservation use case through the payment step revealed two distinct confirmation paths that needed to be reflected as separate methods on Reservation. I added confirmAndAwardPoints() for the credit card path and confirmWithLoyaltyPoints() for the loyalty points redemption path. This in turn required two corresponding methods on Customer — addLoyaltyPoints() to increment the balance on confirmation, and deductLoyaltyPoints() to decrement it when points are redeemed. Both include validation guards: addLoyaltyPoints() rejects non-positive values and deductLoyaltyPoints() throws an exception if the balance is insufficient, preventing it from going negative.

2. Pre-Creation Derived Attribute Calculations

Methods: Tracing the summary step of the reservation flow revealed that /totalPrice and /loyaltyPointsEarned need to be computed before the reservation is persisted, so the customer can preview the cost and points earned. This led to the addition of two static methods on Reservation — calculateTotalPrice() and calculateLoyaltyPointsEarned() — which compute the values from raw parameters without requiring a saved object.

After completing the Make a Reservation use case, I performed dynamic analysis a few more use cases and applied the findings directly to the design class diagram without producing separate scenarios or activity diagrams.